

• سوال ۱: آپشن -c در دستور gcc چه کاری انجام می‌دهد؟ خروجی آن چه فایلی است و چه کاربردی دارد؟

آپشن -c - چه کاری انجام میدهد؟ 🤔

این آپشن مرحله Linking را حذف میکند. یعنی سورس کد رو فقط تا مرحله Object Code پیش میبره 🤖

خروجی آن چه فایلی است؟ 🤔

یک فایل با پسوند .o (Object File) که حاوی کد ماشینِ جابجاپذیر (Relocatable Machine Code) است. 🤖

چه کاربردی دارد؟ 🤔

Programming ← اجازه میده هر بخش از پروژه را جداگانه کامپایل کنیم

Speed ← در صورت تغییر در یک فایل ، نیاز به کامپایل مجدد کل پروژه نیست (فقط همان فایل کامپایل میشه)

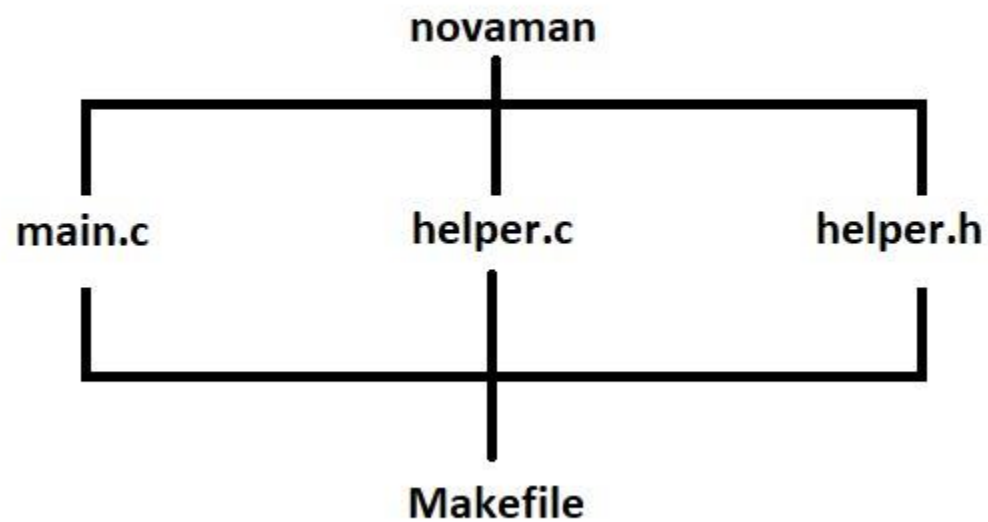
Building the library ← برای ساختن کتابخانه های استاتیک (.a) یا داینامیک (.so) ابتدا باید فایل های .o را داشته باشیم

• سوال ۲: در gdb، تفاوت اصلی بین دستور next و step چیست؟ چه زمانی از هر کدام استفاده می‌کنیم؟

next	اجرای خط بعدی بدون ورود به تابع	از next وقتی استفاده می‌کنیم که فقط بخواهیم جریان برنامه را سریع‌تر دنبال کنیم و وارد جزئیات داخلی تابع نشویم
step	اجرای خط بعدی با ورود به تابع	از step وقتی استفاده می‌کنیم که بخواهیم رفتار داخلی تابع را بررسی کنیم

- **سوال ۳:** یک پروژه کوچک C با دو فایل ایجاد کنید: `main.c` و `helper.c`. تابع `main` در فایل اول، یک تابع را از فایل دوم صدا می‌زند. یک `Makefile` بنویسید که هر دو فایل را به آبجکت فایل (o) کامپایل کرده و سپس آن‌ها را به یکدیگر لینک کنید تا فایل اجرایی نهایی ساخته شود.

ساختار پروژه کوچکی به نام `novaman`



- **سوال ۳:** یک پروژه کوچک C با دو فایل ایجاد کنید: `main.c` و `helper.c`. تابع `main` در فایل اول، یک تابع را از فایل دوم صدا می‌زند. یک `Makefile` بنویسید که هر دو فایل را به آبجکت فایل (o) کامپایل کرده و سپس آن‌ها را به یکدیگر لینک کنید تا فایل اجرایی نهایی ساخته شود.

قدم اول : فایل `helper.c`

```
#include <stdio.h>

void say_hello(void) {

    printf("Hello from helper.c\n");

}
```

- سوال ۳: یک پروژه کوچک C با دو فایل ایجاد کنید: `main.c` و `helper.c`. تابع `main` در فایل اول، یک تابع را از فایل دوم صدا می‌زند. یک `Makefile` بنویسید که هر دو فایل را به آبجکت فایل (o) کامپایل کرده و سپس آن‌ها را به یکدیگر لینک کنید تا فایل اجرایی نهایی ساخته شود.

قدم اول : فایل `helper.c`

به کامپایلر می‌گیم تمام ابزارهای مربوط به ورودی و خروجی را که در این فایل تعریف شده‌اند، به برنامه من اضافه کن تا بتوانم از آن‌ها استفاده کنم

```
#include <stdio.h>
```

این فایل فقط تابع `say_hello` را تعریف کرده است

```
void say_hello(void) {
```

ورودی و خروجی نداره

```
printf("Hello from helper.c\n");
```

```
}
```

- **سوال ۳:** یک پروژه کوچک C با دو فایل ایجاد کنید: `main.c` و `helper.c`. تابع `main` در فایل اول، یک تابع را از فایل دوم صدا می‌زند. یک `Makefile` بنویسید که هر دو فایل را به آبجکت فایل (o) کامپایل کرده و سپس آن‌ها را به یکدیگر لینک کنید تا فایل اجرایی نهایی ساخته شود.

قدم دوم : مشکل اصلی ! ...

```
int main(void) {  
  
    say_hello();  
  
    return 0;  
}
```

- **سوال ۳:** یک پروژه کوچک C با دو فایل ایجاد کنید: `main.c` و `helper.c`. تابع `main` در فایل اول، یک تابع را از فایل دوم صدا می‌زند. یک `Makefile` بنویسید که هر دو فایل را به آبجکت فایل (o) کامپایل کرده و سپس آن‌ها را به یکدیگر لینک کنید تا فایل اجرایی نهایی ساخته شود.

حالا در `main.c` می‌خواهیم این تابع را صدا بزنیم :

```
int main(void) {
```

```
    say_hello();
```

```
    return 0;
```

```
}
```

ولی کامپایلر وقتی داره `main.c` رو کامپایل می‌کنه ، هنوز نمیدونه `say_hello` چیه

1

2

پس باید به کامپایلر بگوییم :

نگران نباش کامپایلر جان ! یک تابعی به اسم `say_hello` وجود داره تعریف اصلی‌اش در فایل دیگری هست

3

برای این کار معمولاً از فایل `header` استفاده می‌کنیم

- **سوال ۳:** یک پروژه کوچک C با دو فایل ایجاد کنید: `main.c` و `helper.c`. تابع `main` در فایل اول، یک تابع را از فایل دوم صدا می‌زند. یک `Makefile` بنویسید که هر دو فایل را به آبجکت فایل (o) کامپایل کرده و سپس آن‌ها را به یکدیگر لینک کنید تا فایل اجرایی نهایی ساخته شود.

قدم سوم : فایل `helper.h`

```
#ifndef HELPER_H  
  
#define HELPER_H  
  
void say_hello(void);  
  
#endif
```


- سوال ۳: یک پروژه کوچک C با دو فایل ایجاد کنید: `main.c` و `helper.c`. تابع `main` در فایل اول، یک تابع را از فایل دوم صدا می‌زند. یک `Makefile` بنویسید که هر دو فایل را به آبجکت فایل (o) کامپایل کرده و سپس آن‌ها را به یکدیگر لینک کنید تا فایل اجرایی نهایی ساخته شود.

این دستورات از نوع **Pre-processor** (پیش‌پردازنده) هستند، یعنی قبل از اینکه کد کامپایل بشه، توسط کامپایلر بررسی می‌شن:

#ifndef HELPER_H

#define HELPER_H

وقتی کامپایلر برای بار اول به فایل `helper.h` می‌رسه:

A. چک می‌کنه که آیا `HELPER_H` تعریف شده؟ (جواب: خیر)

B. پس وارد بلاک میشه

C. بلافاصله `HELPER_H` را تعریف می‌کنه

D. محتویات فایل (تعریف توابع) رو می‌خونه

وقتی کامپایلر برای بار دوم (از طریق یک فایل دیگر) به `helper.h` می‌رسه:

A. چک می‌کنه که آیا `HELPER_H` تعریف شده؟ (جواب: بله، چون بار اول تعریفش کردیم)

B. پس کامپایلر از کل محتویات فایل پرش می‌کنه و مستقیم به بعد از `#endif` میره

نتیجه: محتویات فایل فقط یک بار خوانده می‌شود و جلوی خطاهای تکرار گرفته می‌شود

- **سوال ۳:** یک پروژه کوچک C با دو فایل ایجاد کنید: `main.c` و `helper.c`. تابع `main` در فایل اول، یک تابع را از فایل دوم صدا می‌زند. یک `Makefile` بنویسید که هر دو فایل را به آبجکت فایل (o) کامپایل کرده و سپس آن‌ها را به یکدیگر لینک کنید تا فایل اجرایی نهایی ساخته شود.

در فایل **header** فقط معرفی تابع را می‌نویسیم (بدنه‌اش رو نمی‌نویسیم)

```
#ifndef HELPER_H  
  
#define HELPER_H  
  
void say_hello(void);  
  
#endif
```

- **سوال ۳:** یک پروژه کوچک C با دو فایل ایجاد کنید: `main.c` و `helper.c`. تابع `main` در فایل اول، یک تابع را از فایل دوم صدا می‌زند. یک `Makefile` بنویسید که هر دو فایل را به آبجکت فایل (o) کامپایل کرده و سپس آن‌ها را به یکدیگر لینک کنید تا فایل اجرایی نهایی ساخته شود.

قدم چهارم : فایل `main.c`

```
#include "helper.h"

int main(void) {

    say_hello();

    return 0;
}
```

- سوال ۳: یک پروژه کوچک C با دو فایل ایجاد کنید: `main.c` و `helper.c`. تابع `main` در فایل اول، یک تابع را از فایل دوم صدا می‌زند. یک `Makefile` بنویسید که هر دو فایل را به آبجکت فایل (`.o`) کامپایل کرده و سپس آن‌ها را به یکدیگر لینک کنید تا فایل اجرایی نهایی ساخته شود.

حالا در `main.c` فایل `header` را `include` می‌کنیم:

```
#include "helper.h"

int main(void) {

    say_hello();

    return 0;
}
```

یعنی `main.c` از طریق `helper.h` می‌فهمه تابع `say_hello` وجود داره

- سوال ۳: یک پروژه کوچک C با دو فایل ایجاد کنید: `main.c` و `helper.c`. تابع `main` در فایل اول، یک تابع را از فایل دوم صدا می‌زند. یک `Makefile` بنویسید که هر دو فایل را به آبجکت فایل (`.o`) کامپایل کرده و سپس آن‌ها را به یکدیگر لینک کنید تا فایل اجرایی نهایی ساخته شود.

قدم پنجم : ایجاد Makefile

```
app: main.o helper.o
    gcc main.o helper.o -o app

main.o: main.c helper.h
    gcc -c main.c -o main.o

helper.o: helper.c helper.h
    gcc -c helper.c -o helper.o

clean:
    rm -f *.o app
```

- سوال ۳: یک پروژه کوچک C با دو فایل ایجاد کنید: main.c و helper.c. تابع main در فایل اول، یک تابع را از فایل دوم صدا می‌زند. یک Makefile بنویسید که هر دو فایل را به آبجکت فایل (o) کامپایل کرده و سپس آن‌ها را به یکدیگر لینک کنید تا فایل اجرایی نهایی ساخته شود.

برای ایجاد Makefile و نوشتن محتوای درون آن از nano استفاده می‌کنیم

```
app: main.o helper.o
    gcc main.o helper.o -o app

main.o: main.c helper.h
    gcc -c main.c -o main.o

helper.o: helper.c helper.h
    gcc -c helper.c -o helper.o

clean:
    rm -f *.o app
```

خط‌هایی که با gcc یا rm شروع می‌شوند باید با Tab شروع بشن ، نه Space

- سوال ۳: یک پروژه کوچک C با دو فایل ایجاد کنید: `main.c` و `helper.c`. تابع `main` در فایل اول، یک تابع را از فایل دوم صدا می‌زند. یک `Makefile` بنویسید که هر دو فایل را به آبجکت فایل (`.o`) کامپایل کرده و سپس آن‌ها را به یکدیگر لینک کنید تا فایل اجرایی نهایی ساخته شود.



- سوال ۳: یک پروژه کوچک C با دو فایل ایجاد کنید: `main.c` و `helper.c`. تابع `main` در فایل اول، یک تابع را از فایل دوم صدا می‌زند. یک `Makefile` بنویسید که هر دو فایل را به آبجکت فایل (o) کامپایل کرده و سپس آن‌ها را به یکدیگر لینک کنید تا فایل اجرایی نهایی ساخته شود.

اجرای نهایی در ترمینال

Write inside the project folder :

